

Lab exercises

Track 16.6 –  AI on AWS with Amazon Bedrock · 16-ai-tech-stack/tracks/aws-bedrock/lab/EXERCISES.md

Lab — AI on AWS with Amazon Bedrock

Work top to bottom. Each exercise states a **deliverable**; if you cannot produce it, you have not finished. Exercises 1–5 need **no AWS account**. Exercises 6–8 do, and will cost you real money — a few dollars if you are careful, more if you are not. Set a budget alarm first.

1. Read the shapes until they are boring (45 min) — no AWS

Run both scripts in `../code/`.

- **1a.** In `bedrock_simulator.py`, change the tool-loop request so the `toolResult` message has `"role": "assistant"` instead of `"user"`. Run it. Quote the exact `ValidationException` message and explain in one sentence why the service insists on `user`.
- **1b.** Set `inferenceConfig={"maxTokens": 5}` on the plain Converse demo. What `stopReason` comes back? Name one user-visible bug that ships when code ignores it.
- **1c.** In the streaming demo, `json.loads()` each `toolUse` fragment as it arrives instead of accumulating. Paste the traceback. State the rule this proves.
- **1d.** Without looking: write out the `ConverseStream` event order, including which event carries `stopReason` and which carries `usage`. Then check.

Deliverable: four short answers, one per part, with the literal error text where an error was produced.

2. Implement the tool loop from scratch (2h) — no AWS

Do **not** copy `bedrock_simulator.py`. Write a new file, `my_tool_loop.py`, against the `FakeBedrockRuntime` class imported from the simulator.

Your loop must handle:

- An assistant turn containing a `text` block **and** a `toolUse` block.
- **Two** `toolUse` blocks in one assistant message (parallel tool calls) — both results returned in a **single** user message.
- A tool that raises, returned as `"status": "error"`, with the model given a chance to recover.
- A hop limit that terminates cleanly and reports why.

Checks: - A `pytest` file with at least four tests, one per bullet above. - Your loop never mutates messages in a way that loses the assistant's text block. Assert that. - Append a *third* tool to `TOOL_CONFIG` whose description

overlaps the first. Write one paragraph on what you would change about the descriptions to make the model's choice unambiguous.

3. The cross-Region IAM trap (1h) — no AWS

You are deploying with the inference profile `us.anthropic.claude-sonnet-4-5-20250929-v1:0` from `us-east-1`. Your colleague ships this policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "bedrock:InvokeModel",
      "Resource": "arn:aws:bedrock:us-east-1:123456789012:inference-profile/us.anthropic.claude-sonnet-4-5-20250929-v1:0"
    }
  ]
}
```

It works in staging. In production it throws `AccessDeniedException` on roughly one request in three.

- **3a.** Explain the failure in two sentences.
- **3b.** Write the corrected policy. Use the `bedrock:InferenceProfileArn` condition so the model permissions cannot be used for direct calls.
- **3c.** The security team has an SCP that allowlists `us-east-1` only. Write the SCP modification that keeps CRIS working **without** broadly allowing the other Regions for every service.
- **3d.** The same application also streams. What breaks, and what is the one-line fix?

Deliverable: two policy documents and two paragraphs. Reference:

`../code/iam_and_guardrails_examples.md` §1, §3.

4. Cost model with your own assumptions (1.5h) — no AWS

Take a real or plausible workload of your own and build a spreadsheet or script.

Required inputs, each written down explicitly: requests/day · input tokens/request · what fraction of input is a stable prefix · output tokens/request · expected cache-hit rate · current published rates, **fetches from** <https://aws.amazon.com/bedrock/pricing/> **today** for a specific model in a specific Region.

Required outputs: 1. Daily and annual cost, no caching. 2. Daily and annual cost, with caching, showing cache-read and cache-write token counts separately. 3. The same workload with 70% of traffic routed to a smaller model. State the quality assumption you are making and how you would test it. 4. Break-even: how many requests/day before Provisioned Throughput could plausibly win? State every unknown you had to guess — you will find there are several, and that is the finding.

Checks: - Your token arithmetic uses `inputTokens + cacheRead + cacheWrite`, not `inputTokens` alone. Show the two numbers side by side and explain the gap. - Your document says which prices you looked up, and on what date.

Deliverable: the model, plus one paragraph titled *"the number I am least sure about"*.

5. Design the guardrail, then argue with it (1.5h) — no AWS

Pick a domain with real stakes: a healthcare intake assistant, a bank's support chat, an HR policy bot.

- **5a.** Write the full `CreateGuardrail` configuration JSON. Every filter strength, every denied topic, every PII action must have a one-line justification in a comment or companion table.
- **5b.** Write five prompts that **should** be blocked and five that look dangerous but must **not** be — the false-positive set. This is the harder half and the one people skip.
- **5c.** Your `GROUNDING` threshold is 0.75. Describe the experiment that would tell you whether 0.75 is right. Name the metric and the dataset. Cross-check with `../..../15-ai-evals/`.
- **5d.** Your agent calls a tool that fetches a public web page. Explain, in three sentences, why the guardrail you just designed will not protect you, and write the code that fixes it.

Deliverable: the JSON, the ten-prompt table with expected outcomes, and the two paragraphs.

6. First real call, done properly (1h) — costs money

Set an AWS Budget alert before you start. Then:

1. Enable model invocation logging to a CloudWatch log group **before** your first call, with `largeDataDeliveryS3Config` configured.
2. Create an IAM role with the least-privilege policy from Exercise 3b — nothing broader.
3. Run `converse_api_reference.py` in live mode against a small model.
4. Find your own call in the log group.

Checks: - Your log entry contains `identity.arn` and the `requestMetadata` you set. - Run this CloudWatch Logs Insights query and get a non-empty result: `fields identity.arn as principal, input.inputTokenCount as inTokens | stats sum(inTokens) as totalInput, count() as calls by principal` - Deliberately call a model ID you have **not** granted. Record the exact error code and HTTP status. Confirm your retry logic did **not** retry it.

Deliverable: the log entry (redacted), the query output, and the error transcript.

7. Managed RAG vs your own (3h) — costs money

Take ~50 documents you actually care about.

- **7a.** Build a Knowledge Base with **default** chunking on S3 Vectors or OpenSearch Serverless. Ask 20 questions through `Retrieve`. Record hit rate at k=5 by hand.
- **7b.** Rebuild with **hierarchical** chunking. Same 20 questions. Record the delta. Note how often you get back fewer results than `numberOfResults` requested, and explain why.
- **7c.** Compare `RetrieveAndGenerate` against `Retrieve` plus your own generation prompt on the same 20 questions. Which failures does each produce?

- **7d.** Enable a guardrail with contextual grounding on the generation path. How many of your 20 answers get blocked? Are any of those blocks wrong?

Checks: - A 20-row table: question, default hit@5, hierarchical hit@5, R&G answer quality (1–5), your-prompt answer quality (1–5), grounding blocked (y/n). - One paragraph: *"I would / would not build my own RAG pipeline for this corpus, because..."* — with a reason from your data, not from this track.

Deliverable: the table and the paragraph. Then **delete the knowledge base and the vector store**. OpenSearch Serverless bills whether you query it or not.

8. Capstone: the design document (3h)

Produce the artifact from the track's Exit check. One page — genuinely one page — containing:

1. An ASCII reference architecture naming every VPC endpoint you need, and why each one is there.
2. The complete IAM policy JSON, cross-Region destinations included.
3. A cost estimate with assumptions stated, linking the pricing page rather than quoting a rate as fact.
4. The guardrail configuration, each threshold justified.
5. A "when it breaks" section covering: `us-east-1` throttled, the model deprecated, a guardrail false-positive spike, an agent loop that will not terminate.
6. An explicit "we chose Bedrock over X because..." paragraph, where X is either the provider API direct or SageMaker. Argue the *other* side in one sentence too — if you cannot, you have not understood the trade.

Check: hand it to an engineer who has never used Bedrock. If they can find the model ID, the guardrail version, and the failover plan without asking you a question, you passed. If they ask "what happens if this model goes away?", and your document does not answer it, you did not.

9. Stretch: prove a claim in this track wrong (open-ended)

Everything here was verified against `docs.aws.amazon.com` in September 2026. AWS ships weekly. Pick three claims — a model ID, a chunking option, a service tier, the Agents Classic status, an endpoint name — and check them against the current docs.

Deliverable: a short list of what changed, with doc URLs. Open a pull request against this track. Being the person who keeps the docs honest is a better habit than being the person who memorised them.